Multi-Agent Equity Research Terminal

UMD Agentic AI Challenge 2026 - Reproducibility & Usage Guide

Team Name: **Agent Builders**

Gregory Seleznev Siddhant Tiwari Joshua Rose
Jonathan Hughes Leon Gabrielian

April 15, 2026

1 Introduction

A typical S&P 500 company produces a 10,000-word earnings transcript each quarter. An analyst covering 8–15 names must read, cross-reference, and compare all of them under tight deadlines. Existing AI tools solve pieces of this – summarizers condense single transcripts, sentiment models score tone, chatbots answer one-off questions – but none produce a unified output matching how a fundamental analyst thinks: What drove results? What is management guiding toward? What themes cut across the peer set? How does the company compare to competitors segment by segment?

We built a multi-agent system that automates this first-pass workflow end to end. Given a ticker, it identifies peers, retrieves transcripts and financials, and produces a structured report with four sections: an executive summary with metrics and guidance, bullish/bearish/neutral signal detection, cross-company industry trends, and a competitive landscape table comparing segment revenue across the target and peers.

The stack is n8n (workflow orchestration), Pinecone (vector retrieval), xAI Grok (multi-agent reasoning), and a React frontend.

2 System Architecture

Four n8n workflows coordinate through webhooks. A parent orchestrator handles peer selection, a parallel executor manages concurrency, and two sub-workflows (transcript agent, peer agent) run simultaneously.

2.1 Parent Workflow (Orchestrator)

The React frontend POSTs a ticker to the parent webhook. A Grok agent calls the GitHub API to check which companies have data available and selects the three closest peers – candidates must share at least two major revenue segments, belong to the same sector, and have a comparable business model. It returns a JSON object with the target and three peer tickers. A JavaScript node generates unique request IDs, then the parent calls the parallel execution sub-workflow with the ticker, peers, and IDs.

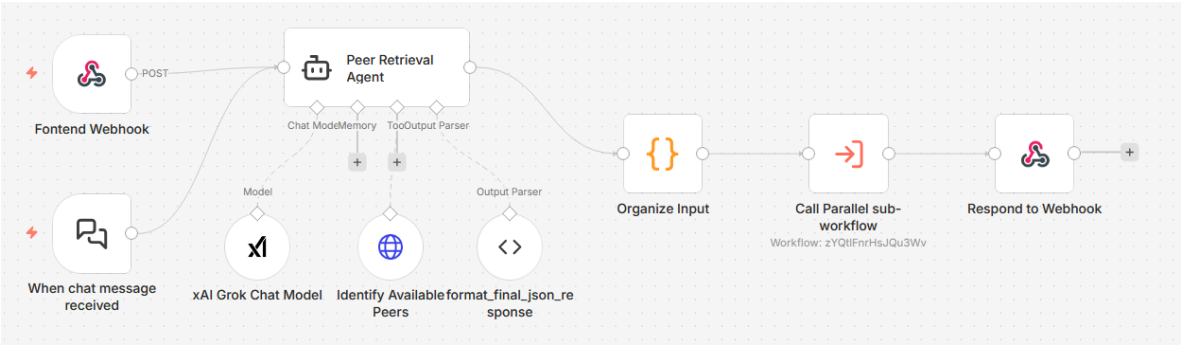

Figure 1: Parent Workflow (Orchestrator)

2.2 Parallel Execution Pattern

The executor fires both sub-workflows via HTTP POST, passing each a callback URL (the execution's resume URL). It then enters a wait loop: each callback updates a tracking set and stores output, and an IF node checks whether both have reported in. If not, it loops back. Retry logic (3-second delay) handles the race condition when both finish near-simultaneously.

Once both complete, a Grok 3 Mini merge agent combines the two outputs into a single JSON matching the frontend schema. It does no rewriting – just structural merging of the transcript agent's output (executive_summary, key_insights) with the peer agent's output (industry_trends, competitive_landscape). The result returns to the parent, which responds to the frontend.

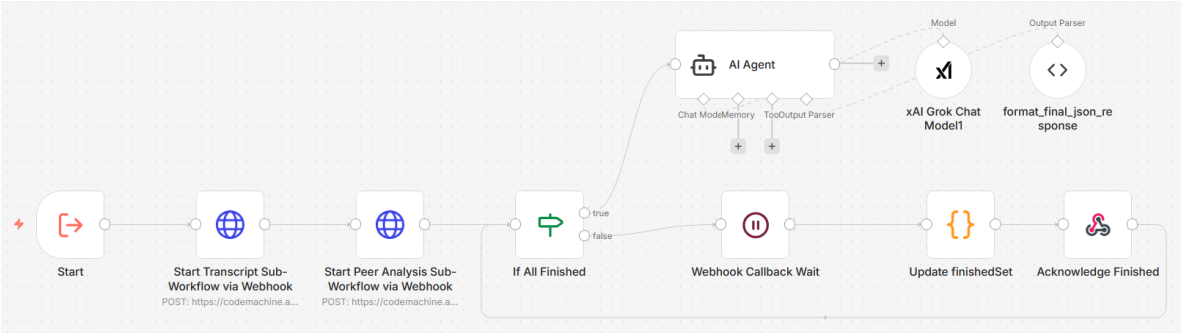

Figure 2: Parallel Execution Pattern

2.3 Transcript Sub-Workflow

Receives a ticker; produces the executive summary and key insights. On trigger it immediately acknowledges (so the executor does not block), then runs asynchronously.

The Grok 4-1 Fast Reasoning agent has two tools: (1) a Pinecone retrieval tool that queries transcript chunks filtered by ticker (top 10 chunks); (2) an HTTP tool that fetches the income statement JSON from GitHub for exact revenue, net income, operating income, and segment figures. A structured output parser enforces the schema (beat/miss/inline, bullish/bearish/neutral). On completion, an HTTP node calls back to the executor.

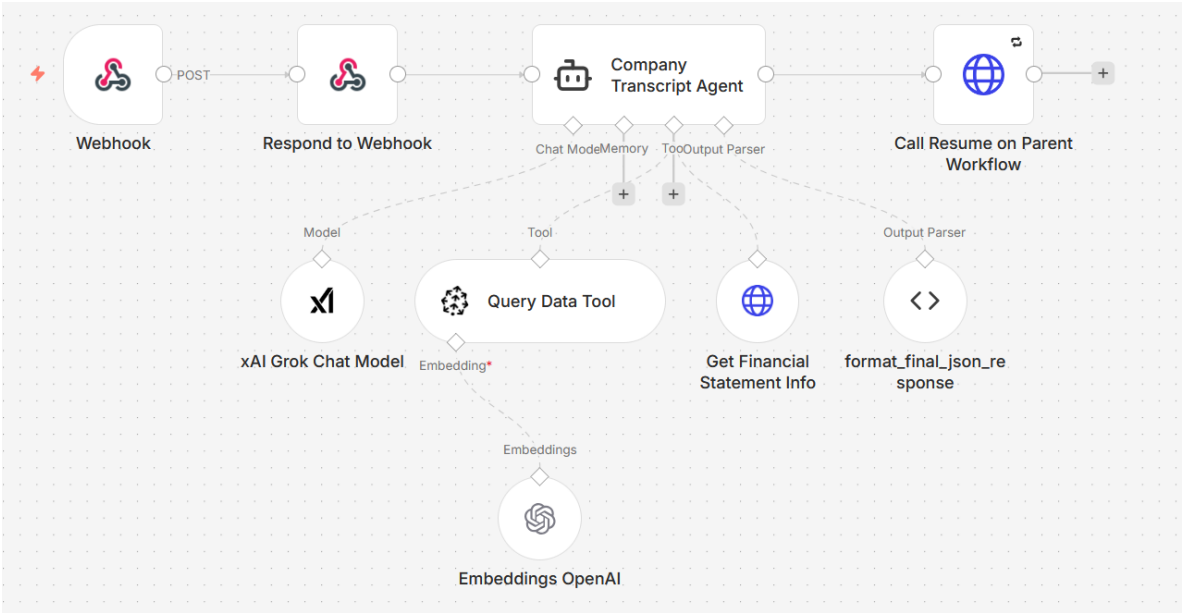

Figure 3: Transcript Sub-Workflow

2.4 Peer Sub-Workflow

Receives the target ticker and peer list; produces industry trends and the competitive landscape table. Acknowledges and runs asynchronously.

The Grok 4-1 Fast Reasoning agent has two tools: (1) a Pinecone retrieval tool for peer transcript chunks; (2) an HTTP tool that fetches income statement JSON from GitHub for every company (target + all peers), one call per ticker.

For **industry trends**, the agent analyzes all peer transcripts together – not per-company summaries – looking for shared language, common tailwinds/headwinds, and divergences. Output: exactly three themes (dominant, emerging, or persistent) with data points and ticker attribution.

For the **competitive landscape**, the agent maps shared segments across all companies using the income statement data. It identifies segments appearing in at least two companies (e.g., Cloud Services mapping AWS to Google Cloud), extracts exact revenue for the two most recent fiscal years, and computes YoY growth. Every company appears with data in at least one segment. Output: a companies array (target first) and offerings array with per-company segment name, revenue, prior-year revenue, YoY growth, and a has_segment flag.

Calls back to the executor on completion.

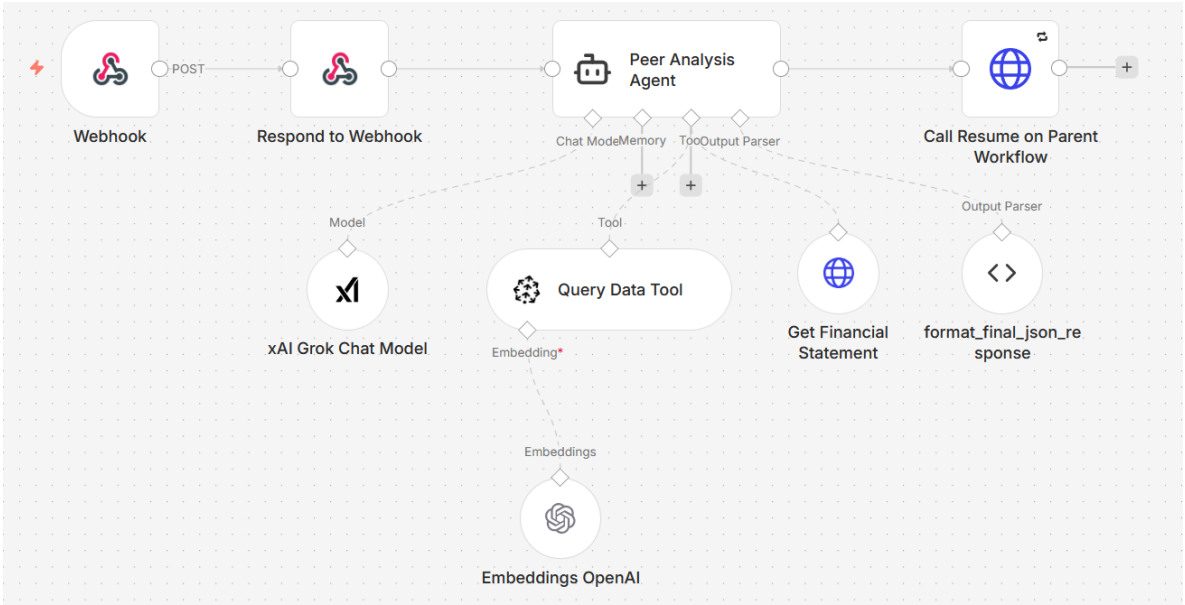

Figure 4: Peer Sub-Workflow

3 Data Pipeline

Two data sources: earnings transcripts (vector embeddings in Pinecone) and financial statements (JSON on GitHub). Both are prepared via Jupyter notebooks before the workflows run.

3.1 Transcript Ingestion

Transcripts come from a public HuggingFace dataset (defeatbeta/yahoo-finance-data), queried via DuckDB. A chunking pipeline: (1) filters boilerplate disclaimers; (2) splits at the prepared-remarks / Q&A boundary by detecting the Operator's "first question" cue; (3) chunks to 400–600 tokens – prepared remarks split per speaker turn at sentence boundaries, Q&A split per exchange (analyst question + management response). Metadata: ticker, quarter, year, date, section, speaker. Chunking runs in `getdata.ipynb`; upsert to Pinecone (index: dbmain, OpenAI embeddings) runs in `push_transcripts_to_pinecone_db.ipynb` via an n8n webhook. Current coverage: AMZN, GOOG, META for FY2025.

3.2 Financial Statement Ingestion

Income statements are pulled from SEC EDGAR via edgartools. For each ticker, the script extracts the XBRL-tagged income statement from the latest 10-K and converts it to JSON (revenue, net income, operating income, segments, three fiscal years). Files are committed to GitHub and served at runtime via raw.githubusercontent.com – no auth or rate limits.

3.3 Runtime Data Access

Transcripts: Pinecone semantic search filtered by ticker metadata (up to 10 chunks per query).
Financials: HTTP GET of raw JSON from GitHub. Both agents (transcript and peer) fetch financials; the peer agent fetches for all companies to build the competitive landscape. Unstructured data stays in the vector store; structured data stays as clean JSON.

4 Demo Walkthrough

Video: this link. The frontend is a React terminal. Enter a ticker, click Run, and the system returns:

1. **Executive Summary** – four metrics (Total Revenue, Net Income, Operating Margin, top two growth segments) with YoY change, a headline takeaway with bolded drivers, primary driver, and forward guidance covering next-quarter targets, CapEx, and segment outlook.
2. **Key Insights** – 4–6 signal cards (bullish/bearish/neutral). For AMZN: AWS acceleration and custom chip scaling (bullish), CapEx pressure and LEO satellite costs (bearish), units growth (neutral).
3. **Industry Trends** – three cross-company themes from peer transcripts, categorized dominant/emerging/persistent, each listing which tickers discussed it.
4. **Competitive Landscape** – comparison table: target and peers as columns, shared segments as rows. Each cell shows the company's segment name, current and prior-year revenue, and YoY growth. Figures come from SEC EDGAR data. Companies without a comparable segment show "No Data."

5 Reproducibility

Code, data, and workflows: github.com/gselez6761/agentic-ai-challenge-data.
Frontend: github.com/gselez6761/agentic-ai-challenge.

1. **API Keys.** Obtain keys for xAI Grok, OpenAI (embeddings only), and Pinecone. Create a Pinecone index named `dbmain` matching OpenAI's embedding dimensionality.
2. **Data Ingestion.** Run `getdata.ipynb` to pull transcripts from HuggingFace, chunk them, and generate income statement JSONs from EDGAR. Run `push_transcripts_to_pinecone_db.ipynb` to upsert chunks to Pinecone. Commit income statement files to `data/income_statements/`.
3. **n8n Setup.** Import the four workflow JSONs from `workflows/` into an n8n cloud instance (parent orchestrator, parallel executor, transcript agent, peer agent). Add xAI, OpenAI, and Pinecone credentials. Activate all four.
4. **Frontend.** Clone the frontend repo, run `npm install`, set the webhook URL env var to the parent workflow endpoint, run `npm start`.
5. **Test.** Enter `AMZN` and click Analyze. The system selects three peers, runs both agents in parallel, and returns the report in 60-90 seconds.